
UNIVERSITÉ DE YAOUNDÉ I
Faculté des Sciences — Département d'Informatique

**Simulation du Prototype
de Prédiction de Pannes
Électriques avec ROS 2**

Mini-Exposé Technique
Robot Operating System 2 (ROS 2 Jazzy Jalisco)

Mengue Ondoua Claude Kerane 25G2071
Année académique : 2025 – 2026

Table des matières

1	Introduction	2
2	Présentation de ROS 2	2
2.1	Qu'est-ce que ROS 2 ?	2
2.2	Concepts fondamentaux de ROS 2	2
2.2.1	Les Nodes (Nœuds)	2
2.2.2	Les Topics	2
2.2.3	Les Messages	3
3	Architecture Globale de la Simulation	3
4	Simulation du Matériel et Génération des Données	3
4.1	Comment le matériel est simulé	3
4.2	Scénarios de pannes simulés	5
5	Format et Traitement des Données	5
5.1	Format des données générées	5
5.2	Les données sont-elles déjà traitées ?	5
6	Implémentation du Modèle ML dans ROS 2	6
6.1	Oui, le modèle est implémenté dans ROS 2	6
7	Gestion du Basculement par ROS 2	7
7.1	Comment le basculement est géré	7
8	Outils ROS 2 Utilisés	8
9	Avantages de cette Approche	8
10	Conclusion	9

1 Introduction

Notre projet vise à concevoir un système intelligent de prédiction de pannes électriques avec basculement automatique de source d'énergie. Suite aux recommandations de notre encadreur, nous adoptons une approche entièrement simulée grâce à **ROS 2** (Robot Operating System 2), ce qui nous permet de reproduire fidèlement le comportement du prototype physique sans nécessiter l'achat de composants électroniques.

Objectif de ce document : Présenter comment ROS 2 est utilisé pour simuler notre dispositif, générer les données d'entraînement, implémenter le modèle de Machine Learning, et gérer le basculement automatique entre sources d'énergie.

2 Présentation de ROS 2

2.1 Qu'est-ce que ROS 2 ?

ROS 2 (Robot Operating System 2) est un **framework open-source** de développement logiciel pour systèmes robotiques et cyber-physiques. Contrairement à son nom, ce n'est pas un système d'exploitation mais plutôt un **middleware** — une couche logicielle qui facilite la communication entre différents composants d'un système distribué.

Dans notre contexte, ROS 2 nous permettra de simuler les composants physiques de notre dispositif (capteurs, relais, microcontrôleur ESP32) sous forme de programmes logiciels communiquant entre eux.

2.2 Concepts fondamentaux de ROS 2

ROS 2 repose sur trois concepts clés qu'il est essentiel de comprendre :

2.2.1 Les Nodes (Nœuds)

Un **node** est un programme indépendant qui réalise une tâche précise. Dans notre projet, chaque composant physique sera simulé par un node dédié :

Composant physique	Node ROS 2	Rôle
Capteur ZMPT101B	<code>tension_sensor_node</code>	Génère les valeurs de tension AC
Capteur ACS712	<code>courant_sensor_node</code>	Génère les valeurs de courant AC
ESP32 (traitement)	<code>prediction_node</code>	Calcule features + prédit la panne
Relais SRD-05VDC	<code>relay_node</code>	Simule le basculement de source
LCD 16×2	<code>display_node</code>	Affiche l'état du système
Module SD + RTC	<code>logger_node</code>	Enregistre les données en base

2.2.2 Les Topics

Un **topic** est un canal de communication nommé sur lequel les nodes échangent des données. Un node peut **publier** (envoyer) des messages sur un topic, et d'autres nodes peuvent **s'abonner** (recevoir) ces messages.

Voici les topics utilisés dans notre projet :

Topic	Publié par	Lu par
/sensor/tension	tension_sensor_node	prediction_node, logger_node
/sensor/courant	courant_sensor_node	prediction_node, logger_node
/prediction/status	prediction_node	relay_node, display_node
/relay/status	relay_node	display_node, logger_node
/system/alert	prediction_node	relay_node, display_node

2.2.3 Les Messages

Un **message** est la structure de données échangée sur un topic. Nous définirons des messages personnalisés pour notre projet :

Listing 1 – Structure du message /sensor/tension

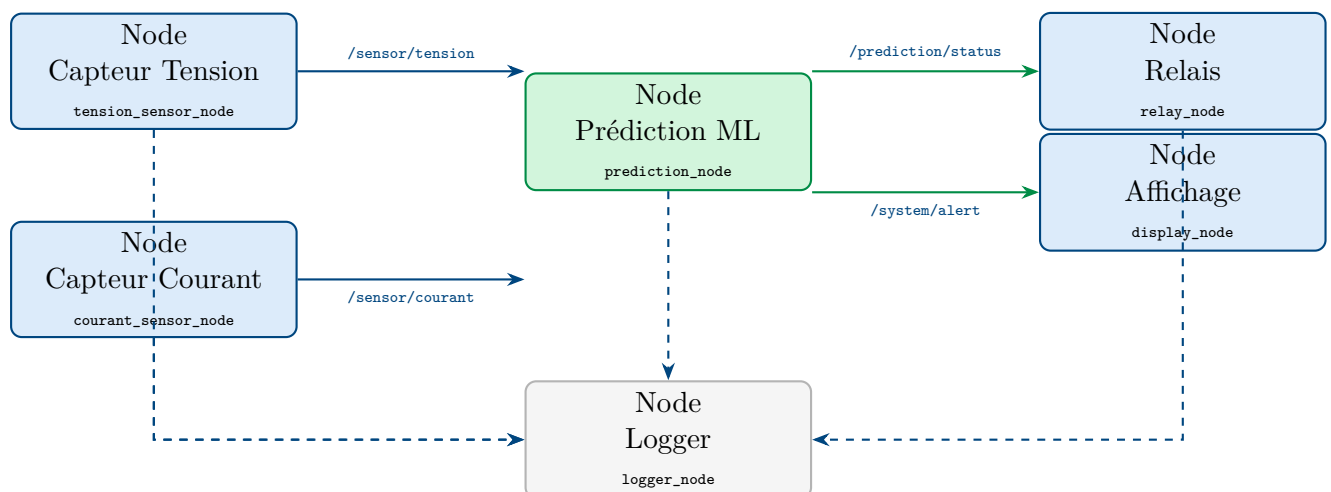
```

1 # Fichier : ElectricalMeasure.msg
2 std_msgs/Header header      # Horodatage
3 float32 valeur              # Valeur mesur e (Volts ou Amp res)
4 float32 valeur_rms          # Valeur efficace (RMS)
5 float32 frequence           # Fr quence du signal (Hz)
6 float32 delta_v             # Variation par rapport la mesure
                             pr c dente
7 string unite                # "V" pour tension, "A" pour courant
8 string source               # "principale" ou "secours"

```

3 Architecture Globale de la Simulation

L'architecture complète de notre simulation ROS 2 est la suivante :



4 Simulation du Matériel et Génération des Données

4.1 Comment le matériel est simulé

Chaque composant physique est remplacé par un nœud ROS 2 qui reproduit son comportement électrique. Le nœud capteur de tension, par exemple, ne lit pas un vrai

ZMPT101B mais génère mathématiquement un signal sinusoïdal à 50 Hz avec des perturbations contrôlées.

Listing 2 – Extrait du node capteur de tension

```

1 import rclpy
2 from rclpy.node import Node
3 import numpy as np
4 import math
5
6 class TensionSensorNode(Node):
7     def __init__(self):
8         super().__init__('tension_sensor_node')
9         self.publisher = self.create_publisher(
10             ElectricalMeasure, '/sensor/tension', 10)
11         # Fréquence d'acquisition : 500 Hz (comme le vrai
12             ESP32)
13         self.timer = self.create_timer(0.002, self.
14             publier_mesure)
15         self.scenario = 'normal' # ou 'baisse', 'surtension',
16             'coupure'
17         self.t = 0.0
18
19     def publier_mesure(self):
20         msg = ElectricalMeasure()
21         msg.header.stamp = self.get_clock().now().to_msg()
22
23         # Signal de base : 220V RMS, 50 Hz
24         v_base = 220.0 * math.sqrt(2) * math.sin(2 * math.pi *
25             50 * self.t)
26
27         if self.scenario == 'normal':
28             bruit = np.random.normal(0, 2.0) # bruit gaussien
29             2V
30             msg.valeur = v_base + bruit
31
32         elif self.scenario == 'baisse':
33             # Descente progressive 220V 170V en 2 secondes
34             facteur = max(0.77, 1.0 - 0.115 * self.t)
35             msg.valeur = v_base * facteur
36
37         elif self.scenario == 'surtension':
38             msg.valeur = v_base * 1.18 # +18% surtension
39
40         elif self.scenario == 'coupure':
41             msg.valeur = 0.0 # Coupure nette
42
43         msg.valeur_rms = abs(msg.valeur) / math.sqrt(2)
44         self.publisher.publish(msg)
45         self.t += 0.002

```

4.2 Scénarios de pannes simulés

Cinq scénarios de pannes sont simulés pour générer un dataset varié et représentatif :

Scénario	Description	Paramètres	Étiquette
Normal	Fonctionnement standard	$V = 220V \pm 5V$, $I = 2A \pm 0.1A$	0 - OK
Baisse tension	Descente progressive	$V : 220V \rightarrow 170V$ en 2s	1 - WARN
Surtension	Pic de tension	$V > 250V$ pendant $< 200ms$	1 - WARN
Coupure nette	Coupure instantanée	$V = 0V$ en $< 20ms$	2 - FAULT
Surcharge	Surconsommation courant	$I > 3.5A$ progressivement	1 - WARN

5 Format et Traitement des Données

5.1 Format des données générées

Les données sont générées à une fréquence de **500 Hz** (500 mesures par seconde), ce qui correspond exactement à la fréquence d'acquisition prévue pour le vrai ESP32. Chaque mesure est publiée sur les topics ROS 2 et simultanément enregistrée par le `logger_node` dans une base de données **SQLite**.

Le format d'un enregistrement est le suivant :

timestamp	tension_V	courant_A	v_rms	i_rms	freq	label
1711234567	311.12	2.83	219.98	2.001	50.0	0
1711234567	310.89	2.81	219.74	1.989	50.0	0
1711234568	295.43	2.78	208.90	1.966	50.0	1
1711234568	280.12	2.75	198.01	1.944	50.0	1

5.2 Les données sont-elles déjà traitées ?

Non, les données brutes générées ne sont pas directement utilisables par le modèle ML. Le `prediction_node` effectue un **feature engineering** en temps réel sur des fenêtres glissantes de 5 secondes :

Feature	Formule	Description
V_{eff}	$\sqrt{\frac{1}{N} \sum v_i^2}$	Tension efficace (RMS)
I_{eff}	$\sqrt{\frac{1}{N} \sum i_i^2}$	Courant efficace (RMS)
P_{active}	$\frac{1}{N} \sum v_i \cdot i_i$	Puissance active
σ_V	$\sqrt{\frac{1}{N} \sum (v_i - \bar{v})^2}$	Écart-type tension
ΔV	$V_{eff}(t) - V_{eff}(t-1)$	Variation tension
$\Delta^2 V$	$\Delta V(t) - \Delta V(t-1)$	Accélération tension
R_{risque}	$\alpha \Delta V + \beta \sigma_V$	Indicateur de risque

6 Implémentation du Modèle ML dans ROS 2

6.1 Oui, le modèle est implémenté dans ROS 2

Le modèle de Machine Learning est **directement intégré** dans le `prediction_node`. Nous utilisons une **Random Forest** entraînée sur les données générées par la simulation, puis exportée et chargée dans le node.

Le pipeline complet est le suivant :

1. Le `logger_node` accumule les données simulées pendant plusieurs heures
2. Les données sont exportées en CSV et un modèle Random Forest est entraîné avec `scikit-learn` (Python)
3. Le modèle entraîné est exporté au format `.pkl` (pickle) ou ONNX pour une meilleure portabilité
4. Le `prediction_node` charge ce modèle au démarrage et l'utilise pour prédire en temps réel

Listing 3 – Prédiction ML dans le `prediction_node`

```

1  import joblib
2  import numpy as np
3  from sklearn.preprocessing import StandardScaler
4
5  class PredictionNode(Node):
6      def __init__(self):
7          super().__init__('prediction_node')
8          # Chargement du modèle entraîné
9          self.model = joblib.load('random_forest_smart_power.pkl')
10         self.scaler = joblib.load('scaler_smart_power.pkl')
11
12         # Abonnements aux capteurs
13         self.sub_tension = self.create_subscription(
14             ElectricalMeasure, '/sensor/tension',
15             self.callback_tension, 10)
16         self.sub_courant = self.create_subscription(
17             ElectricalMeasure, '/sensor/courant',
18             self.callback_courant, 10)
19
20         # Publication de la prédiction
21         self.pub_prediction = self.create_publisher(
22             PredictionStatus, '/prediction/status', 10)
23
24         self.fenetre_tension = [] # Buffer 2500 points (5s
25                                     500Hz)
26         self.fenetre_courant = []
27
28     def predire(self):
29         # Calcul des features sur la fenêtre glissante
30         features = self.calculer_features()
31         features_norm = self.scaler.transform([features])

```

```

31
32     # Prediction
33     prediction = self.model.predict(features_norm)[0]
34     proba = self.model.predict_proba(features_norm)[0]
35
36     msg = PredictionStatus()
37     msg.label = int(prediction)
38     msg.probabilité_panne = float(proba[1])
39     msg.action = 'BASCULER' if prediction >= 1 else 'NORMAL'
40     self.pub_prediction.publish(msg)

```

7 Gestion du Basculement par ROS 2

7.1 Comment le basculement est géré

Le basculement automatique entre la source principale et la source de secours est géré par le `relay_node`. Ce node s'abonne au topic `/prediction/status` et déclenche le basculement selon la logique suivante :

Règle de basculement :

- Si `label = 0` (NORMAL) → Source principale active
- Si `label = 1` (WARN) et probabilité > 70% → Basculement préventif
- Si `label = 2` (FAULT) → Basculement immédiat
- Retour source principale : stable depuis 30 secondes consécutives

La séquence **break-before-make** est respectée avec un délai de 50ms entre l'ouverture du relais 1 et la fermeture du relais 2, pour éviter tout court-circuit entre les deux sources.

Listing 4 – Logique de basculement dans `relay_node`

```

1  class RelayNode(Node):
2      def __init__(self):
3          super().__init__('relay_node')
4          self.source_active = 'principale'
5          self.compteur_stabilite = 0
6
7          self.sub = self.create_subscription(
8              PredictionStatus, '/prediction/status',
9              self.callback_prediction, 10)
10         self.pub = self.create_publisher(
11             RelayStatus, '/relay/status', 10)
12
13         def callback_prediction(self, msg):
14             if msg.label >= 1 and self.source_active == 'principale':
15                 self.get_logger().warn('PANNE DETECTEE
16                                     Basculement vers secours')
17                 self.basculer_vers_secours()

```



```

17
18     elif msg.label == 0 and self.source_active == 'secours'
19         :
20         self.compteur_stabilite += 1
21         if self.compteur_stabilite >= 15000: # 30s      500
22             Hz
23             self.get_logger().info('Retour source
24                 principale')
25             self.basculer_vers_principale()
26             self.compteur_stabilite = 0
27
28     def basculer_vers_secours(self):
29         # S quence break-before-make : 50ms de d lai
30         self.relais_1 = False # Ouverture source principale
31         import time; time.sleep(0.05)
32         self.relais_2 = True # Fermeture source secours
33         self.source_active = 'secours'
34         self.publier_statut()

```

8 Outils ROS 2 Utilisés

ROS 2 fournit plusieurs outils en ligne de commande très utiles pour notre projet :

Outil	Utilisation dans notre projet
ros2 run	Lancer un node individuel
ros2 launch	Lancer tous les nodes simultanément
ros2 topic echo	Afficher en temps réel les données publiées sur un topic
ros2 bag record	Enregistrer tous les messages pour analyse ultérieure
ros2 bag play	Rejouer un enregistrement (rejouer une simulation)
rqt_plot	Visualiser graphiquement les données de tension/courant
rqt_graph	Visualiser le graphe de communication entre nodes
Gazebo	Simulateur 3D optionnel pour visualiser le prototype

9 Avantages de cette Approche

1. **Aucun coût matériel** — tout est simulé logiciellement
2. **Reproductibilité** — les mêmes scénarios peuvent être rejoués à l'infini grâce à ros2 bag play
3. **Données maîtrisées** — on contrôle exactement les types de pannes générées
4. **Scalabilité** — on peut simuler des semaines de données en quelques heures

5. **Débogage facilité** — `rqt_plot` permet de visualiser les données en temps réel
6. **Transition vers le réel** — si le matériel est disponible plus tard, il suffit de remplacer les nodes simulés par les vrais drivers de capteurs

10 Conclusion

ROS 2 constitue un cadre idéal pour simuler notre dispositif de prédiction de pannes électriques. Grâce à son architecture basée sur des nodes communicants via des topics, nous pouvons reproduire fidèlement le comportement de chaque composant physique — capteurs, microcontrôleur, relais — et générer des données d’entraînement réalistes pour notre modèle de Machine Learning.

Cette approche nous permet de mener à bien notre projet sans contrainte financière, tout en développant des compétences avancées en Data Science, en traitement du signal et en systèmes distribués.